Supplementary Material I

Note 1:

Adaptive MMD Kernel Bandwidth Selection in KPCA-DA: The Median Pairwise-Distance Heuristic (auto_gamma)

Background:

The domain alignment step in KPCA-DA minimizes a Maximum Mean Discrepancy (MMD) loss between source and target gene embeddings. The MMD is computed via a Radial Basis Function (RBF) kernel:

$$k(x, y) = exp(-\gamma \cdot ||x - y||^2) \quad \text{(N1E1)}$$

where $\gamma > 0$ is the kernel bandwidth parameter. The choice of γ directly determines whether the optimization can produce meaningful gradient signals: if γ is too small relative to the actual inter-domain distances, the kernel evaluations approach zero and all gradients vanish, preventing the optimizer from reducing the MMD regardless of the number of training steps.

In KPCA-DA, source and target datasets are each projected into a low-dimensional space by fitting separate KernelPCA models. Because the two decompositions are independent, the resulting embeddings may occupy different regions of the embedding space with large inter-domain pairwise distances.

The Median Pairwise-Distance Heuristic

The median heuristic is a principled, data-driven method for setting the RBF kernel bandwidth. It selects γ such that the kernel evaluates to approximately 0.5 at the median pairwise distance, placing the kernel in its most informative regime — neither saturated nor near-zero.

In KPCA-DA, the heuristic is applied to the initial KPCA embeddings of the common-gene subsets of both domains, before the optimization loop begins. Let $Z_s \in \mathbb{R}^{(G' \times d)}$ and $Z_t \in \mathbb{R}^{(G' \times d)}$ denote the source and target embeddings restricted to the G'=common gene set. The pooled point set is:

$$Z_pool = [\, Z_s \,;\, Z_t \,] \in \mathbb{R}^{(2G' \times d)} \quad \text{(N1E2)}$$

The squared pairwise distances are computed across all distinct point pairs:

$$D^2_{ij} = ||z_pool,i - z_pool,j||^2, \quad i \neq j \quad \text{(N1E3)}$$

The adaptive bandwidth is then set as:

$$\gamma^* = 1 / median\{ D^2_{ij} \} \quad \text{(N1E4)}$$

γ is computed once from the pooled embeddings of the whole source domain and the 70% target adaptation subset, prior to the KPCA-DA's evaluation on the held-out 30% target-test set. This

ensures bandwidth selection has no access to test data, consistent with the held-out protocol described in the main text's Experimental Setup.

This value is computed once per run from the pre-optimization embeddings and replaces any manually specified mmd_gamma for the duration of training. The procedure adds negligible overhead (a single $O((2G')^2)$ distance matrix computation) and requires no additional hyperparameter choices.

Implementation:

The adaptive bandwidth is implemented in kpcada.py as the function compute_median_gamma (Zs, Zt) and is controlled by the auto_gamma parameter available in fit_transform (), KPCA_DA, and tune_lambda().

The following PyTorch implementation is excerpted from kpcada.py

```
Z_pool = torch.cat([Zs, Zt], dim=0)
dists_sq = torch.cdist(Z_pool, Z_pool) ** 2
median_sq = torch.median(dists_sq[dists_sq > 0])
gamma = (1.0 / median_sq.clamp(min=1e-8)).item()
```

The parameter auto_gamma defaults to True in all API entry points. To use a manually specified bandwidth instead, set auto_gamma=False and provide the desired value via mmd_gamma.

Results:

Applying equation N1E4 to the TCGA → GSE45498 task yields $\gamma^* = 965.55$. As an independent check, Note 2 reports a two-dimensional grid search spanning $\gamma \in \{500, 1000, 2000, 5000, 10000\}$ and $\lambda \in \{10^{-4}, \ldots, 1.0\}$ (Tables S2–S3). The lowest MMD loss across this grid occurs at $\gamma = 1000$ — the grid points closest to the heuristic's continuous estimate — confirming that the auto-computed bandwidth falls within the empirically optimal region rather than being an arbitrary or unvalidated choice.

Note 2:

Selection of the Regularization Parameter λ:

Overview:

The KPCA-DA framework aligns gene expression distributions across sequencing cohorts by minimizing a composite objective in the kernel PCA embedding space. A key hyperparameter, the regularization coefficient λ, balances domain alignment against structural preservation of the target manifold. This supplementary note describes the theoretical role of λ, the automated bandwidth selection strategy introduced to ensure effective optimization, and the empirical results of a grid search with the MMD loss and total loss.

Optimization Objective

At each gradient step the following loss is minimized with respect to the adapted target embedding $Z_t \in \mathbb{R}^{(G \times d)}$:

$$\mathscr{L}(\lambda) = \mathscr{L}_{mmd} + \lambda \cdot \mathscr{L}_{reg} \qquad \text{(N2E1)}$$

$\mathscr{L}_{mmd}$ measures the distributional divergence between source and target embeddings over the common gene set (G') using Maximum Mean Discrepancy:

$$\mathscr{L}_{mmd} = | K_tt.mean() - 2 \cdot K_st.mean() | \qquad \text{(N2E2)}$$

where $K_tt = k(Z_t, Z_t)$ and $K_st = k(Z_s, Z_t)$ are RBF kernel matrices evaluated on the adapted target embeddings and the common-gene subsets of the source. No source embeddings term (e.g. *K_ss*) is in the eq.(N2E2), as it is constant with respect to Z_t and contributes no gradient during target-only optimization. $\mathscr{L}_{reg}$ is the structural regularization term:

$$\mathscr{L}_{reg} = tr(Z_t^T K_t Z_t) \qquad \text{(N2E3)}$$

where K_t is the RBF kernel matrix of the original target feature space and $tr(\cdot)$ denotes the matrix trace. This term preserves the intrinsic kernel geometry of the target domain throughout optimization. The coefficient λ governs the balance between the two objectives:

- Small λ: alignment is prioritized; the embedding is free to move toward the source distribution.
- Large λ: structural preservation dominates; the degree of adaptation is constrained.

Selection Procedure

A grid search was conducted over six candidate values: $\lambda \in \{0.0001, 0.001, 0.01, 0.1, 1.0, 10.0\}$. For each candidate, KPCA-DA was run with 500 SGD steps (learning rate 0.001), RBF kernel. Evaluation was applied focusing on MMD loss and total loss $\mathscr{L}$. No class labels required. Optimality is defined as the λ value that achieves the lowest final $\mathscr{L}_{\text{mmd}}$, reflecting the tightest distributional alignment between source and target embeddings. The λ grid search (Tables S1–S3) was conducted using the same 70% target adaptation subset described in Note 1; the held-out 30% target-test set was excluded from all selection procedures for both γ and λ.

Results:

Table S1 reports the MMD loss and the total loss at different value of **λ** and the γ is here is 965.55 calculated automatically.

Table S1. λ grid search results — MMD loss and total loss (Protocol A).

λ	Final MMD Loss	Final Total Loss
0.0001	0.000263	0.000812
0.001★	0.000101	0.001187
0.01	0.000223	0.001672
0.1	0.000473	0.103291
1.0	0.000932	0.289571
10.0	0.002253	0.282827

★ *Best λ by lowest final MMD loss . All values obtained with γ =965.55, epochs=500, lr=0.001.*

Table S2 reports the final MMD loss for each (γ, λ) combination. Each value is the MMD between the source gene embeddings and the adaptation target gene embeddings after 500 epochs of optimization.

	$\lambda = 0.0001$	**$\lambda = 0.001$**	**$\lambda = 0.01$**	**$\lambda = 0.1$**	**$\lambda = 1.0$**
$\gamma = 500$	0.000261	0.000565	0.000327	0.000403	0.001488
$\gamma = 1000$	**0.001782**	**0.000112**	**0.000192**	**0.000348**	**0.003495**
$\gamma = 2000$	0.000538	0.000666	0.002378	0.001588	0.003801
$\gamma = 5000$	0.003507	0.000778	0.000193	0.000596	0.003144
$\gamma = 10000$	0.003461	0.000769	0.004893	0.006737	0.000152

Table S2. Final MMD loss for each (γ, λ) combination after 500 epochs. Lower values indicate better domain alignment. Bold row: $\gamma = 1000$, which yields the lowest MMD at $\gamma = 0.001$.

Table S3 reports the corresponding total loss $\mathscr{L}(\lambda) = \mathscr{L}_{mmd} + \lambda \cdot \mathscr{L}_{reg}$ for each combination.

	$\lambda = 0.0001$	**$\lambda = 0.001$**	**$\lambda = 0.01$**	**$\lambda = 0.1$**	**$\lambda = 1.0$**
$\gamma = 500$	0.001675	0.001099	0.012630	0.142538	0.404146
$\gamma = 1000$	**0.002544**	**0.000668**	**0.016675**	**0.091729**	**0.287634**
$\gamma = 2000$	0.001205	0.001778	0.012127	0.126424	0.192166
$\gamma = 5000$	0.000887	0.002007	0.012475	0.077277	0.233047
$\gamma = 10000$	0.005713	0.003656	0.012256	0.069652	0.133804

Table S3. Final total loss for each (γ, λ) combination. The total loss grows monotonically with λ, confirming that higher λ increasingly weights the regularization term over alignment.

From table S1, $\lambda = 0.001$ with the automatically selected γ achieved the lowest final MMD (0.000101), indicating the tightest distributional alignment between the TCGA and GSE45498 embeddings. Following the grid search, the optimal pair of *(γ, λ)* is $\gamma = 965.55$ and $\lambda = 0.001$. Here, $\gamma = 1000$ is near the median-heuristic value of 965.55, placing the MMD kernel in its optimal sensitivity range, while $\lambda = 0.001$ applies minimal regularization pressure, allowing the alignment objective to dominate the optimization as intended.

Practical Recommendation:

auto_gamma=True should be used by default for all KPCA-DA runs. As Tables S2–S3 show, the correct bandwidth cannot be reliably guessed. For the TCGA → GSE45498 task, the heuristic returns $\gamma^* = 965.55$, which the grid search independently confirms lies within this optimal region. Once the bandwidth is fixed by auto_gamma, λ should be selected with a one-dimensional sweep over a small log-spaced grid (e.g., $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$), choosing the value that

minimizes the final loss. For this task, $\lambda = 0.001$ was optimal; performance degraded once $\lambda \geq 1$, where the regularization term increasingly dominates the objective and constrains adaptation (Table S3). For TCGA → Desmedt scenario the auto_gamma calculated was $\gamma = 20.44$ and optimal was $\lambda = 0.0001$. For CPTAC → SMC scenario the values selected were $\gamma = 18.17$ and $\lambda = 1.0$.

For new datasets, we recommend enabling auto_gamma and running the λ sweep over the mentioned range. The full two-dimensional grid in Table S2 is provided as a validation check on the heuristic, not as a step required for routine use — since the auto-computed γ already lands in the optimal region, re-running the full grid for every new dataset is unnecessary. Because both hyperparameters are selected using only the 70% adaptation subset, the reported held-out test performance in the main text reflects a fully unsupervised, leakage-free selection procedure.

Note 3:

Data Preprocessing and Feature Selection:

Raw gene expression matrices are provided as samples × genes CSV files. Before image transformation, three sequential preprocessing steps are applied: variance-based gene filtering, min–max normalization, and label encoding.

Variance-Based Gene Filtering

Genes whose expression variance across all training samples falls below the k-th percentile are discarded, retaining the top (100 − k)% most informative features. In the reported experiments k = 30, retaining the top 70% of genes.

$$G_sel = \{ g \in G : Var(g) \geq Q_k(Var(G)) \} \quad \text{(N3E1)}$$

where Q_k denotes the k-th percentile of the variance distribution over the full gene set G.

Min–Max Normalization

Each retained gene is linearly rescaled to the unit interval independently, so that downstream kernel computations are not dominated by high-magnitude features.

$$\tilde{x}_ig = (x_ig - x_min,g) / (x_max,g - x_min,g + \varepsilon) \quad \text{(N3E2)}$$

The scaler fits exclusively on the source (training) partition and is applied without refitting to both source and target, preserving the same feature scale across domains.

Label Encoding

Class labels are encoded as consecutive non-negative integers via scikit-learn LabelEncoder. When source and target datasets carry different label vocabularies, a single encoder is fit on the union of both label sets to guarantee consistent integer mapping.

Converting Probe to Genes for Desmedt dataset:

The Desmedt dataset was originally annotated with microarray probe identifiers rather than gene symbols. To enable cross-dataset gene matching, probe IDs were mapped to HUGO Gene Nomenclature Committee (HGNC) gene symbols using Ensembl BioMart. They were converted to Hugo Symbols for matching feature names across both TCGA and Desmedt using the code given here.

Note 4:

Gradient-Weighted Class Activation Mapping (Grad-CAM) Feature Selection:

After training the ResNet-50 classifier on source domain images, Grad-CAM [1] is used to identify which pixel regions — and therefore which genes — are most discriminative for each class. This produces a class-specific gene subset that is biologically interpretable and transferable to independent datasets.

Class Activation Map Computation:

For each class c, Grad-CAM computes the gradient of the class score y^c with respect to the feature maps A^k of the final convolutional layer, globally average-pooled to obtain per-channel importance weights:

$$\alpha_k^c = (1/Z) \sum\sum \partial y^c / \partial A_{ij}^k \qquad (N4E1)$$

The class activation map is then formed by a ReLU-gated weighted sum over all K feature channels:

$$L_{ij}^c = ReLU(\Sigma_k \alpha_k^c \cdot A_{ij}^k) \qquad (N4E2)$$

Activations are computed over all training samples per class and averaged to yield a single representative map L^c, which is then normalized to [0, 1].

Gene Feature Selection:

A threshold τ is applied to the normalized class activation map. Pixel positions (i, j) where the activation exceeds the threshold are identified, and their corresponding gene indices are recovered via the inverse pixel-to-gene map stored by the ImageTransformer:

$$F_sel^c = \{ g \in G : \bar{L}^c_px(g) > \tau \} \quad (N4E3)$$

where px(g) denotes the pixel position assigned to gene g by the ImageTransformer. The default threshold is $\tau = 0.5$. The resulting per-class gene sets can be saved for downstream biological enrichment analysis.

REFERENCES

[1] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. Proceedings of the IEEE ICCV, 618–626.